



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The image is framed by a thin white border.

COMP-2024 / COMP-2039
**Artificial Intelligence
Methods**

Lecture 4
Move Acceptance

IDENTIFY

- Move Acceptance
- Simulated Annealing
- Parameter Setting Issues



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Local Search Metaheuristics and Move Acceptance Methods



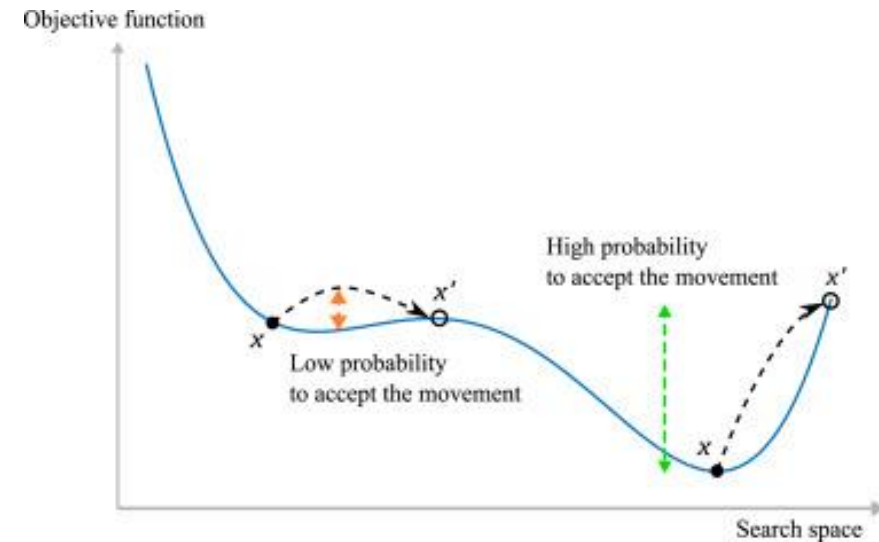
Note

- The slides content and examples in this section are based on minimization problem. Same can be applied to maximization, just reverse the sign.



Move Acceptance

- **Move acceptance** in metaheuristics refers to the *criteria or rules* used to decide whether a potential solution (referred to as a "move") is **accepted** or **rejected** during the search process.
- This concept is critical for balancing **exploration** (searching diverse areas of the solution space) and **exploitation** (refining solutions in promising regions).
- The *choice of move acceptance strategy* significantly influences the performance of a metaheuristic algorithm.





Move Acceptance

- **Move:** A transition from one solution to another in the solution space.
- **Objective Value:** The quality of a solution, typically measured by a evaluation, fitness or cost function.
- **Exploration vs. Exploitation:**
 - Exploration focuses on diversifying the search to avoid getting trapped in local optima.
 - Exploitation aims to refine and improve the current best solutions.



Single Point Based Iterative Search (Local Search Metaheuristics)

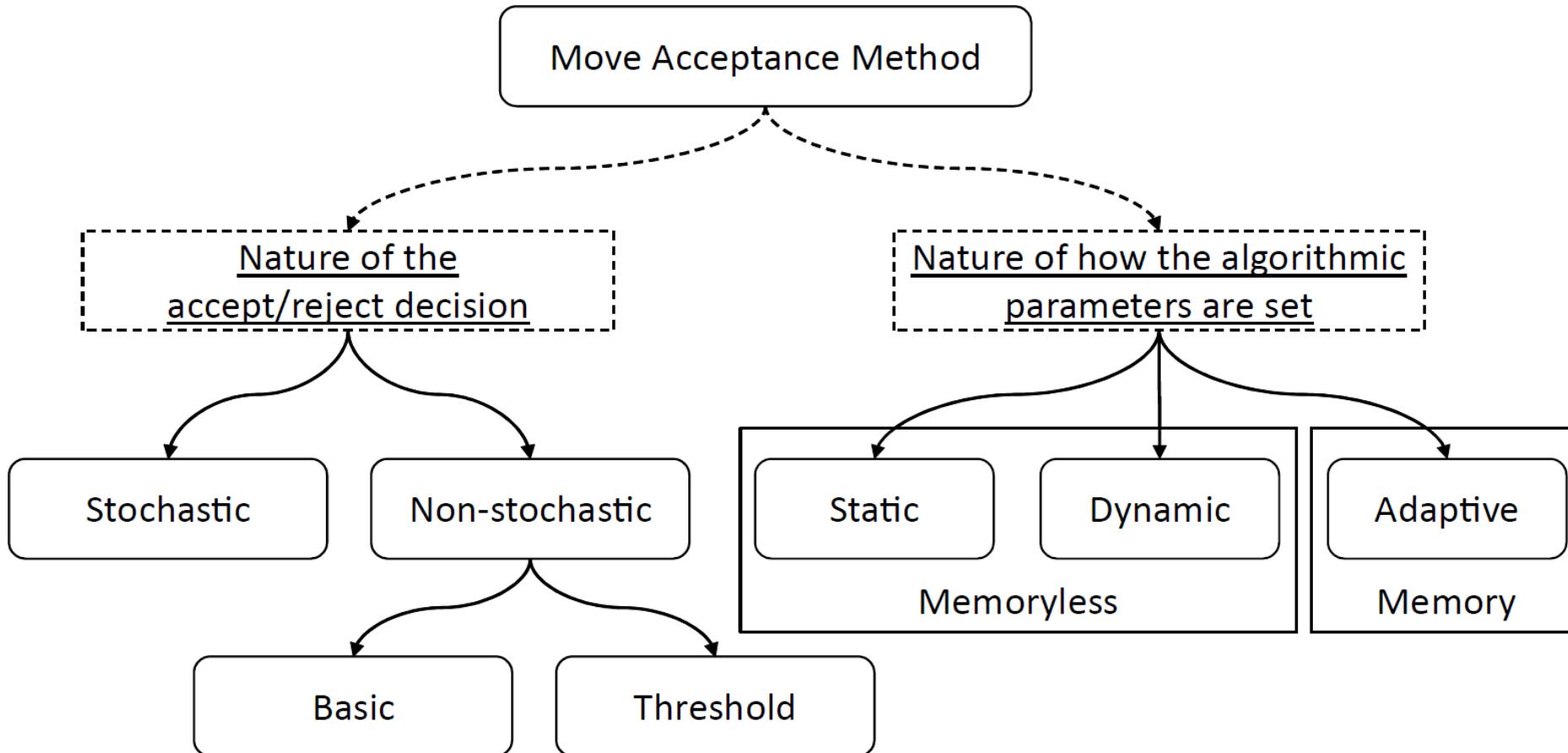
- **Move Acceptance** decides whether to **accept** or **reject** the new solution considering its evaluation/quality, $f(s')$.
- **Accepting non-improving moves** could be used as a mechanism to *escape from local optimum*.

Algorithm 1: Outline of a Local Search Metaheuristic.

```
1  $s \leftarrow \text{generateInitialSolution}();$ 
2  $s_{best} \leftarrow s;$ 
3 while termination criteria not met do
4    $s' \leftarrow \text{apply}(h, s);$ 
5    $\text{process}_1();$ 
6    $s \leftarrow \text{acceptRejectDecision}(s, s');$ 
7   if  $f(s').\text{isBetterThan}(f(s_{best}))$  then
8      $s_{best} \leftarrow s';$ 
9   end
10   $\text{process}_2();$ 
11 end
12 return  $s_{best};$ 
```



Move Acceptance Methods – A Taxonomy



W.G. Jackson, E. Özcan and R. I. John, Move Acceptance in Local Search Metaheuristics for Cross-domain Search, Expert Systems with Applications, 109: 131-151, 2018 [[original PDF](#)]. [[PDF](#)]



Nature of How the Algorithmic Parameters Are Set

- **Memoryless**
 - **Static:** Parameters remain constant throughout the search process.
 - **Dynamic:** Parameters change in response to the search progress, often to balance exploration and exploitation.
- **Memory:** Utilizes past information to influence current decisions.
 - **Adaptive:** Parameters adjust based on previous outcomes, allowing the algorithm to learn and improve its search strategy.



Parameter Setting Mechanisms in Move Acceptance (e.g. Intensity of Mutation (IoM))

- **Static**, either there is no parameter to set or parameters are set to a *fixed value*.
- E.g., IoM=5;
- **Dynamic**, parameter values vary with respect to time/iteration count. Given the same candidate and current solutions at the same current elapsed time or iteration count, the acceptance threshold or acceptance probability would be the same *irrespective of search history*.
- E.g., $\text{IoM} = \text{round}(1 + (\text{iter}_{\text{current}} / \text{iter}_{\text{max}}) * 4)$;



Parameter Setting Mechanisms in Move Acceptance

- **Adaptive**, Given the same candidate and current solutions at the same current elapsed time or iteration count, the acceptance threshold or acceptance probability is not guaranteed to be the same as one or more components *depend on search history*.
- E.g., if for 100 steps best solution found in the stage cannot be improved, then $IoM++$, and after any improvement, reset $IoM=1$;



Nature of the Accept/Reject Decision

- **Stochastic:** Decisions to accept or reject a move are made *probabilistically*, allowing for exploration of the solution space even if the new solution is worse than the current one.
- **Non-stochastic:** Decisions are made *deterministically*, typically based on *a fixed criterion* like improvement in objective function value.
 - **Basic:** Simple criteria for accepting moves, often based on *direct* comparisons of *objective function values*.
 - **Threshold:** More complex criteria that might involve a *threshold value* to determine whether a move is accepted.



Non-stochastic Basic Move Acceptance Methods

- Reuse the objective values of previously encountered solutions for the accept/reject decisions to enhance efficiency and decision-making in metaheuristics.
- Instead of evaluating every solution independently, the algorithm leverages historical data (stored objective values) to make accept/reject decisions.
- This method reduces redundant computations and introduces more structured exploration.



Non-stochastic Basic Move Acceptance Methods

- **Static**
 - **All Moves:** returns true regardless of $f(s')$
 - **Improving Moves Only (minimization):** $f(s') < f(s)$
 - **Improving and Equal (minimization):** $f(s') \leq f(s)$
- **Dynamic:** Dynamic rules adjust criteria based on the algorithm's progress or environmental changes. These are typically not reused but can incorporate state-dependent heuristics.



Non-stochastic Basic Move Acceptance Methods

- **Adaptive (with memory)**
 - **Late Acceptance:** compares the quality of the solution with that of the past solution accepted/visited L iterations previously s_{t-L} , and accepts the move if and only if $f(s') \leq f(s_{t-L})$
 - It introduces a memory-based criterion for deciding whether to accept a new solution by comparing it with the quality of a solution encountered L iterations earlier, rather than the current solution. This approach aims to prevent the search from getting stuck in local optima by introducing a delayed influence of past solutions.



Late Acceptance

- Traditional metaheuristics often compare the quality of a new solution s' with the **current solution** s (e.g., $f(s') \leq f(s)$).
- Late Acceptance modifies this by comparing $f(s')$ with the objective value of a solution $st - L$, visited L iterations ago.
- **Acceptance Rule:** Accept s' if:

$$f(s') \leq f(st - L)$$

where:

- $f(s')$: Objective value of the candidate solution s' .
- $f(st - L)$: Objective value of the solution L iterations prior.



Late Acceptance Algorithm – Pseudocode

- Maintains a *sliding window* of previously visited states and accepts moves that improve the objective function value compared to the state at the end of the window.
- **Initialisation:** Assign all elements of the list to be equal to the initial cost (objective value).
- List for the history of the objective values of the recent solutions is implemented as a *circular queue*.

```
Produce an initial solution  $s$ 
Calculate initial cost function  $C(s)$ 
Specify  $L_h$ 
For all  $k \in \{0 \dots L_h - 1\}$   $f_k := C(s)$ 
First iteration  $I := 0$ ;  $I_{idle} := 0$ 
Do until ( $I > 100000$ ) and ( $I_{idle} > I * 0.02$ )
    Construct a candidate solution  $s^*$ 
    Calculate a candidate cost function  $C(s^*)$ 
    If  $C(s^*) \geq C(s)$ 
        Then increment the idle iterations number  $I_{idle} := I_{idle} + 1$ 
        Else reset the idle iterations number  $I_{idle} := 0$ 
    Calculate the virtual beginning  $v := I \bmod L_h$ 
    If  $C(s^*) < f_v$  or  $C(s^*) \leq C(s)$ 
        Then accept the candidate  $s := s^*$ 
        Else reject the candidate  $s := s$ 
    If  $C(s) < f_v$ 
        Then update the fitness array  $f_v := C(s)$ 
    Increment the iteration number  $I := I + 1$ 
```

Edmund K. Burke, Yuri Bykov, [The late acceptance Hill-Climbing heuristic](#). EJOR 258(1): 70-78 (2017)



Practical Example of Late Acceptance Run

1. Initialize:

- Set L , the length of the history list.
- Store the first L solutions' objective values.

2. Iterate:

- Compute $f(s')$ for a new candidate solution s' .
- Compare $f(s')$ with $f(st - L)$ from the history list.
- Accept s' if $f(s') \leq f(st - L)$; otherwise, reject it.

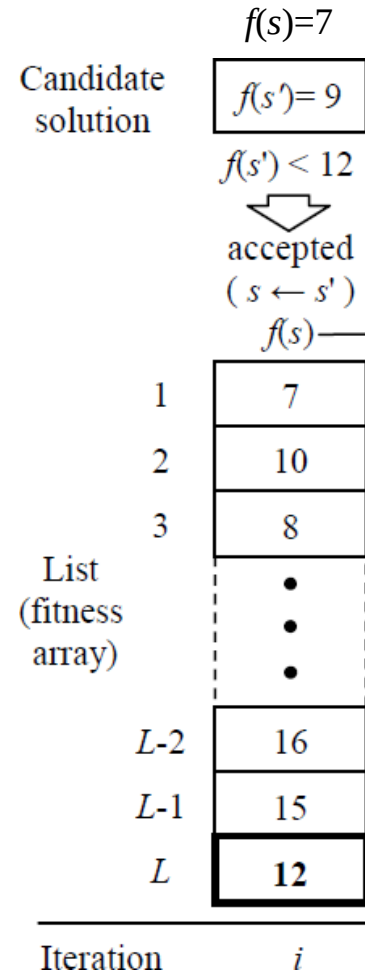
3. Update Memory:

- Add $f(s)$ (current solution's objective value) to the history list.
- Remove the oldest entry if the list exceeds L entries.



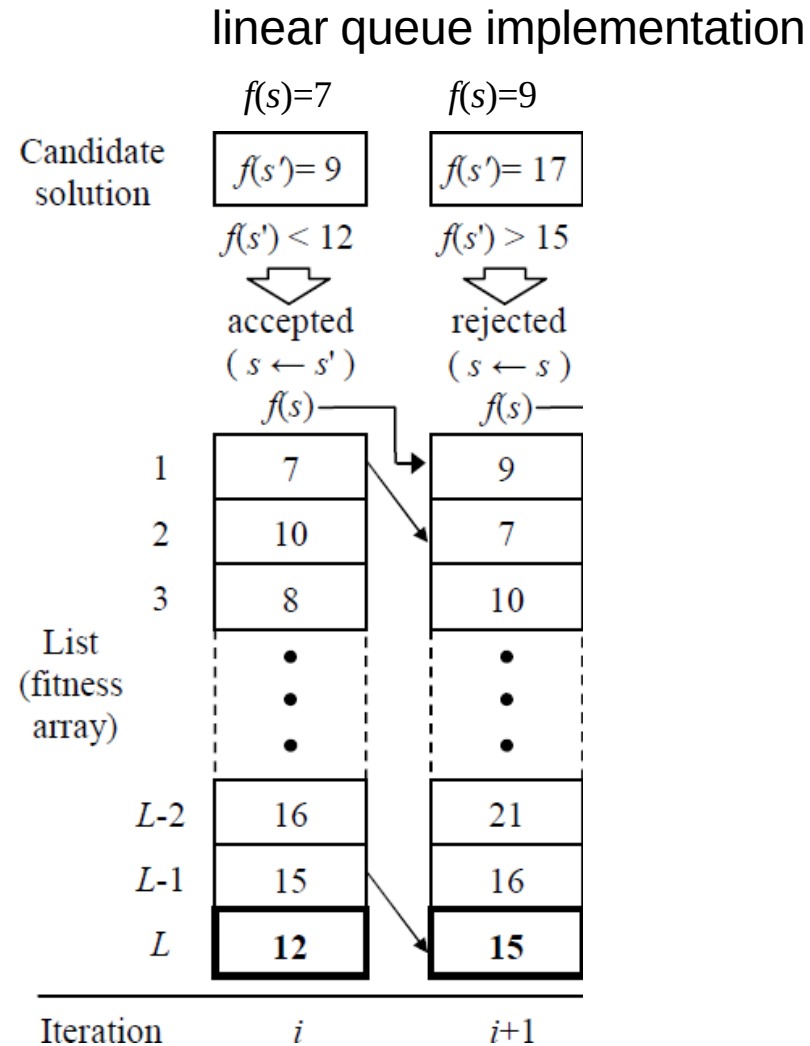
A Sample Run – Late Acceptance Algorithm

linear queue implementation



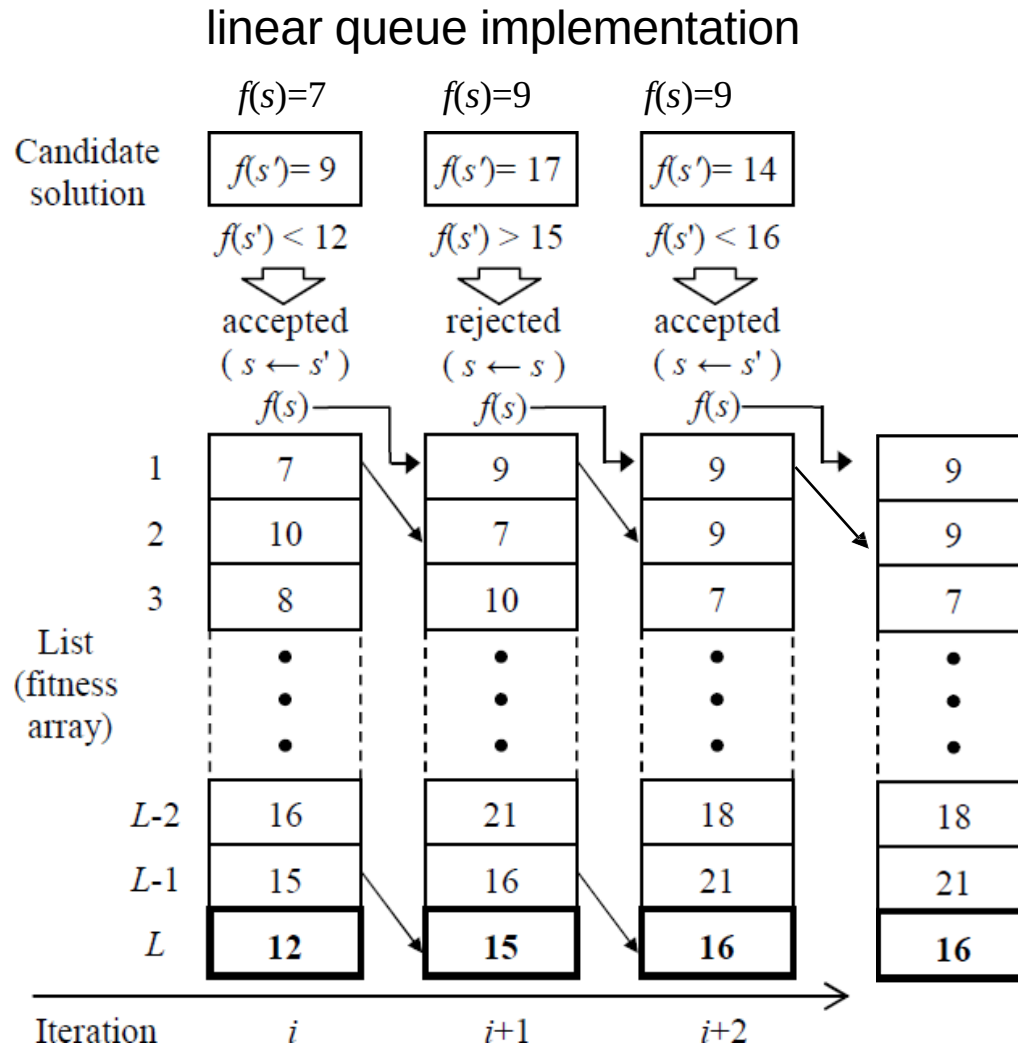


A Sample Run – Late Acceptance Algorithm

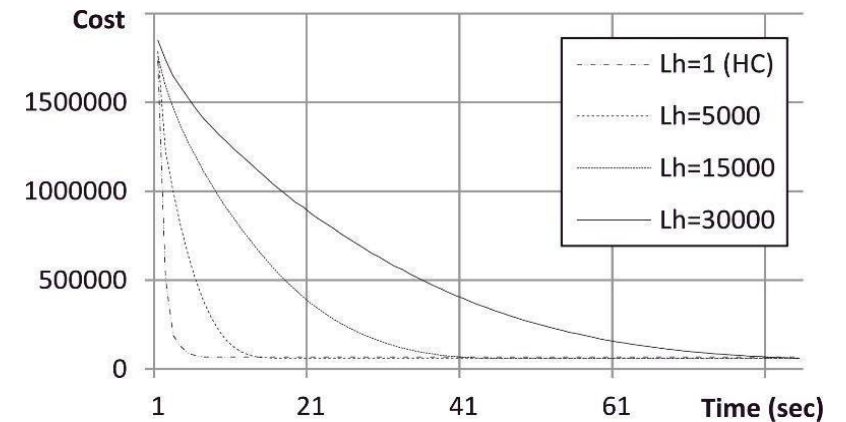




A Sample Run – Late Acceptance Algorithm



- Simple approach with a single parameter, yet the setting of the *list length* parameter influences the overall performance of the algorithm.





Nature of the Accept/Reject Decision

Method	Criteria	Strengths	Weaknesses
All Moves	Accept all moves	Maximizes exploration	No focus on exploitation
Improving Moves Only	$f(s') < f(s)$	Prioritizes exploitation	Gets stuck in local optima
Improving and Equal	$f(s') \leq f(s)$	Balanced between exploration and exploitation	Limited escape from local optima
Late Acceptance	$f(s') \leq f(st - L)$	Balances short-term and long-term memory	Depends on the choice of history length



Non-stochastic **Threshold** Move Acceptance

– minimising F

- Determine a threshold which is *in the vicinity of a chosen solution quality*, e.g., the quality of the best solution found so far or current solution and accept all solutions below that threshold.

```
■  $s_0 = \text{generateInitialSolution};$   
■  $S, s_{best} = S_0;$   
■ // initialise other relevant parameters if there is any  
REPEAT  
    ■  $s' = \text{makeMove}(s, \text{memory});$  // choose a neighbour of  $s^*$   
    ■  $\text{threshold-}\epsilon = \text{moveAcceptance-getThreshold}(s, s', \text{memory});$   
    ■ if (  $f(s') \leq \text{threshold-}\epsilon$  )  $s = s';$  // else reject new solution  $s'$   
    ■  $s_{best} = \text{updateBest } s, s';$  // keep track of  $s_{best}$   
■ UNTIL(termination conditions are satisfied);  
RETURN  $s_{best}$ 
```




Non-stochastic **Threshold** Move Acceptance – Examples

- **Static**

- Accept a worsening solution if the worsening of the objective value is no worse than a *fixed value*.

- **Dynamic**

- Great Deluge
- Flex Deluge

- **Adaptive**

- Record to record travel (RRT)
- Extended Great Deluge
- Modified Great Deluge

[New Optimization Heuristics. The Great Deluge Algorithm and the Record-to-Record Travel. Dueck, Gunter, Journal of Computational Physics, Volume 104, Issue 1, January 1993, Pages 86-92](#)



Dynamic Non-Stochastic Threshold Move Acceptance

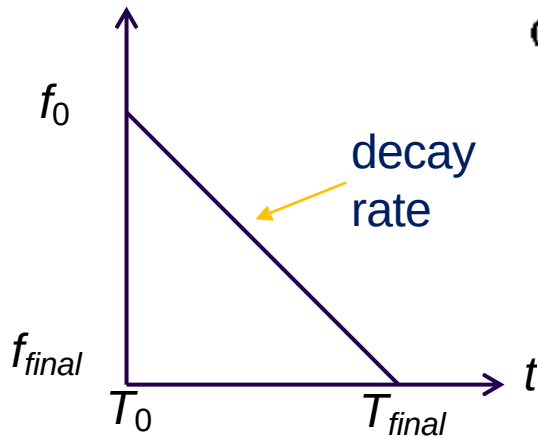
- Great Deluge

- Inspired by the idea of a "rising water level" (**threshold**) in a flooding landscape.
- A move is accepted if the objective value of the candidate solution $f(s')$ is *less than or equal to the current threshold* T (representing the water level).
- Initialize T (threshold) to a large value, typically the objective value of the initial solution. (minimization)
- Over time, decrease T (water level rises), narrowing the acceptance criterion.
- Acceptance Rule: $f(s') \leq T$
- Threshold Update: $T = T - \Delta$ where Δ is the rate of decrease (step size).



Great Deluge – minimisation

Example:



THE GREAT DELUGE ALGORITHM FOR MAXIMIZATION.

Choose an initial configuration

choose the “rain speed” $UP > 0$

choose the initial WATER-LEVEL > 0

Opt: choose a new configuration which is a stochastic
small

perturbation of the old configuration

compute $E := \text{quality}(\text{new configuration}) \rightarrow (E \text{ is } f(.))$

IF $E > \text{WATER-LEVEL}$

THEN old configuration := new configuration

WATER-LEVEL := WATER-LEVEL + UP $\forall N$

IF a long time no increase in quality or too many
iterations

THEN stop

GOTO Opt

decay rate

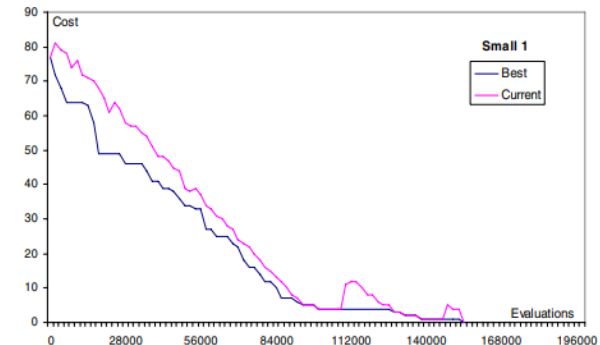
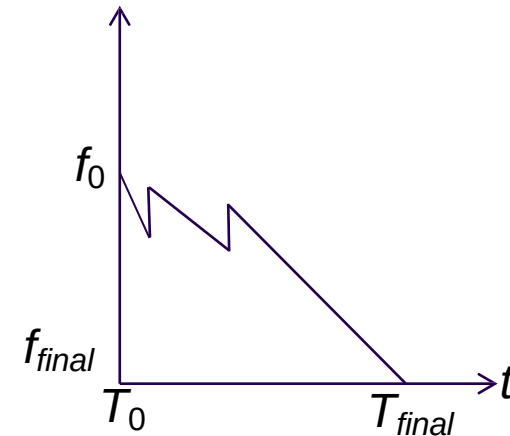
[Gunter Dueck, Tobias Scheuer, Threshold accepting: A general purpose optimization algorithm appearing superior to simulated annealing, Journal of Computational Physics, Volume 90, Issue 1, September 1990, Pages 161-175](#)



Great Deluge (Minimisation)

- Feedback is received during the search and *decay-rate* is updated/ reset accordingly whenever there is no improvement for a *long time*.

```
Set the initial solution  $s$  using a construction heuristic;  
Calculate initial cost function  $f(s)$   
Set Initial Boundary Level  $B_0 = f(s)$   
Set initial decay Rate  $\Delta B$  based on Cooling Parameter  
While stopping criteria not met do  
  Apply neighbourhood Heuristic  $S^*$  on  $S$   
  Calculate  $f(s^*)$   
  If  $f(s^*) \leq f(s)$  or  $(f(s^*) \leq B \text{ Then}$   
    Accept  $s = s^*$   
  Lower Boundary  $B = B - \Delta B$   
  If no improvement in given time  $T$  Then  
    Reset Boundary Level  $B_0 = f(s)$   
    Set new decay rate  $\Delta B$  based on Secondary  
    Cooling Parameter
```



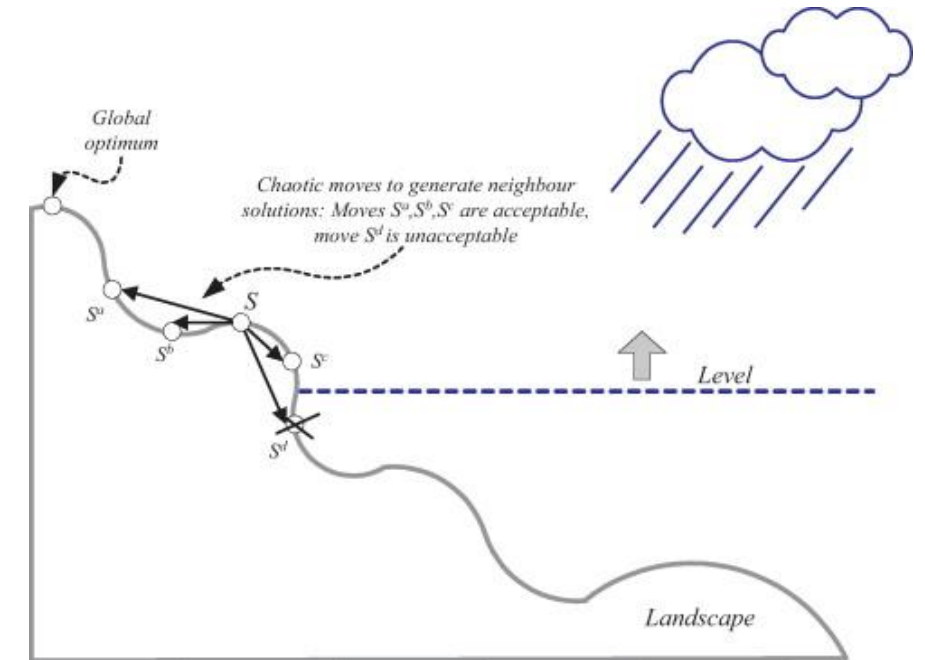
- http://www.cs.nott.ac.uk/~pszajp/pubs/extended_great_deluge_Final.pdf
- https://link.springer.com/content/pdf/10.1007%2F978-3-540-72584-8_71.pdf



Dynamic Non-Stochastic Threshold Move Acceptance

- Flex Deluge

- An **extension of Great Deluge**, introducing more flexibility in how the threshold T is updated. (“flexible great deluge”)
- The threshold T *can increase or decrease dynamically* based on the search's progress or predefined rules.
- Allows for temporary relaxation of the acceptance criterion to escape local optima.
- Problems with *rugged landscapes* or *numerous local optima*.





Adaptive Non-Stochastic Threshold Move Acceptance - Record-to-Record Travel (RRT)

- A *threshold-based* algorithm where the threshold is tied to the best objective value found so far, with an added tolerance ϵ .
- A move is accepted if the objective value is within the *allowable tolerance of the current record (best solution)*.
- Acceptance Rule: $f(s') \leq f^* + \epsilon$
- where:
 - f^* : Best objective value encountered so far (record).
 - ϵ : **Tolerance (can be static or dynamic)**.



Adaptive Non-Stochastic Threshold Move Acceptance - Extended Great Deluge

- Enhances Great Deluge by allowing the threshold T to adapt dynamically *based on search performance or progress*.
- Instead of a fixed decrement Δ (minimization), the rate of change of T depends on feedback from the quality of recent solutions or the distance from the global best.



Adaptive Non-Stochastic Threshold Move Acceptance - Modified Great Deluge

- Another variation of Great Deluge, designed to address specific weaknesses, such as getting stuck in plateaus or being too rigid.
- Introduces *additional rules* or parameters to *modify the threshold update dynamically*.
- For instance (minimization):
 - Allowing T to temporarily increase (relaxing the acceptance criterion).
 - Adjusting Δ based on the improvement rate of the solutions.



Dynamic vs Adaptive Non-Stochastic Threshold Move Acceptance

Method	Threshold Update	Adaptability	Use Case
Great Deluge	Fixed decrement Δ	Low	Simple optimization problems.
Flex Deluge	Dynamic, flexible updates	Medium	Problems with local optima or rugged spaces.
Record-to-Record Travel	Tied to $f^* + \epsilon$	High	Problems needing exploration and exploitation.
Extended Great Deluge	Adaptive based on performance	High	Complex landscapes with varying challenges.
Modified Great Deluge	Dynamic rules and parameters	Very High	Problems with plateaus or slow convergence.



Examples of Metaheuristics with Move Acceptance

- **Tabu Search:** Uses memory-based acceptance with tabu lists. (Lecture 3)
- **Simulated Annealing:** Uses probabilistic (stochastic) acceptance based on temperature (Lecture 4)
- **Genetic Algorithms:** Combines elitism, selection, and diversity-based acceptance. (Lecture 5)



University of
Nottingham

UK | CHINA | MALAYSIA

Stochastic Move Acceptance – Simulated Annealing



Note

- The slides content and examples in this section are based on minimization problem. Same can be applied to maximization, just reverse the sign.



Stochastic Move Acceptance

- **Stochastic Move Acceptance** (SMA) refers to the mechanism by which a solution to an optimization problem is accepted or rejected based on a *probability distribution*, rather than strictly on merit (e.g., whether it improves the objective function).
- This introduces a level of *randomness* that can help the algorithm escape local optima and explore the solution space more effectively.



Stochastic Move Acceptance

- Probability-Based Decisions:
 - Solutions with worse objective values can still be accepted with a certain probability.
 - This prevents the algorithm from getting stuck in local minima.
- Trade-off Between Exploration and Exploitation:
 - By accepting some worse solutions, the algorithm explores the solution space.
 - Gradually shifts towards exploitation by reducing the probability of accepting worse solutions over time.
- **Temperature** or Threshold Control:
 - Many stochastic methods use a "temperature" or "threshold" parameter that decreases over time to reduce randomness and focus on convergence (exploitation).



Stochastic Move Acceptance

```
s0 = generateInitialSolution();  
S, Sbest = s0;  
REPEAT  
    s' = makeMove(s, memory);    // choose a neighbour of s  
    P = moveAcceptance-getAcceptanceProbability(s, s', memory);  
    r = getRandomValue();          // e.g., a uniform random value ∈ [0,1)  
    if ( f(s') isBetterThan( f(s) ) || (r < P) )  
        s = s';                    // else reject new solution s'  
    Sbest ← updateBest s, s' ;    // keep track of Sbest  
UNTIL (termination conditions are satisfied);  
RETURN Sbest;
```



Calculate Acceptance Probability

- `moveAcceptance.getAcceptanceProbability(s, s', memory)`
 - calculates the probability P of accepting the new candidate solution s' compared to the current solution s .
 - This probability may depend on factors such as the change in the objective function and possibly a *temperature parameter* (if it's a *simulated annealing approach*).
- `getRandomValue()` generates a random number r in the range $[0, 1]$, which will be compared against the acceptance probability P .



Decision Making

- The condition checks 2 things:
 - 1. Accept better solution:** If the new solution s' is better than the current solution s (i.e., $f(s') < f(s)$), it is accepted.
 - 2. Accept worse solution:** If s' is not better, it can still be accepted with a probability P (i.e., $r < P$). This introduces the *stochastic* element → add in element of chance to explore and escape from local optima when the temperature is still high in the early stage before cooling



Stochastic Move Acceptance – Examples

- **Static**

- E.g., Naive Acceptance: P is fixed, e.g. if improving $P=1.0$, else $P=0.5$.

- **Dynamic**

- E.g., **Simulated Annealing**: P changes in time with respect to the difference in the quality of current and previous solutions (see the next slides). Temperature parameter is changes dynamically.

- **Adaptive**

- E.g., **Simulated Annealing with reheating**: P is modified via increasing temperature time to time causing partial restart – increasing the probability of acceptance of non-improving solutions.



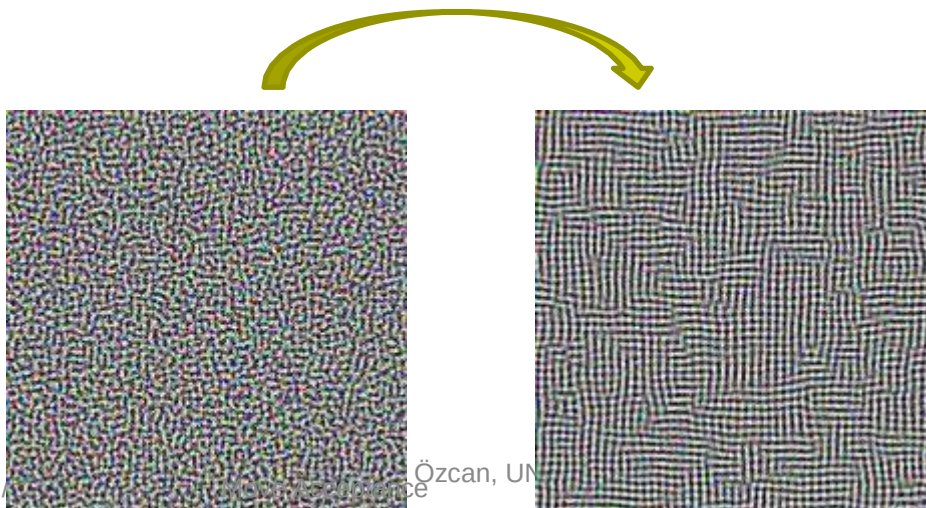
Simulated Annealing

- A **stochastic local search algorithm** inspired by the physical process of annealing (Kirkpatrick et al. 1983).
- Easy to implement.
- Achieves good performance given **sufficient running time**.
- Requires a **good parameter setting** for improved performance.
- Has interesting theoretical properties (**convergence**), but these are of very limited practical relevance.



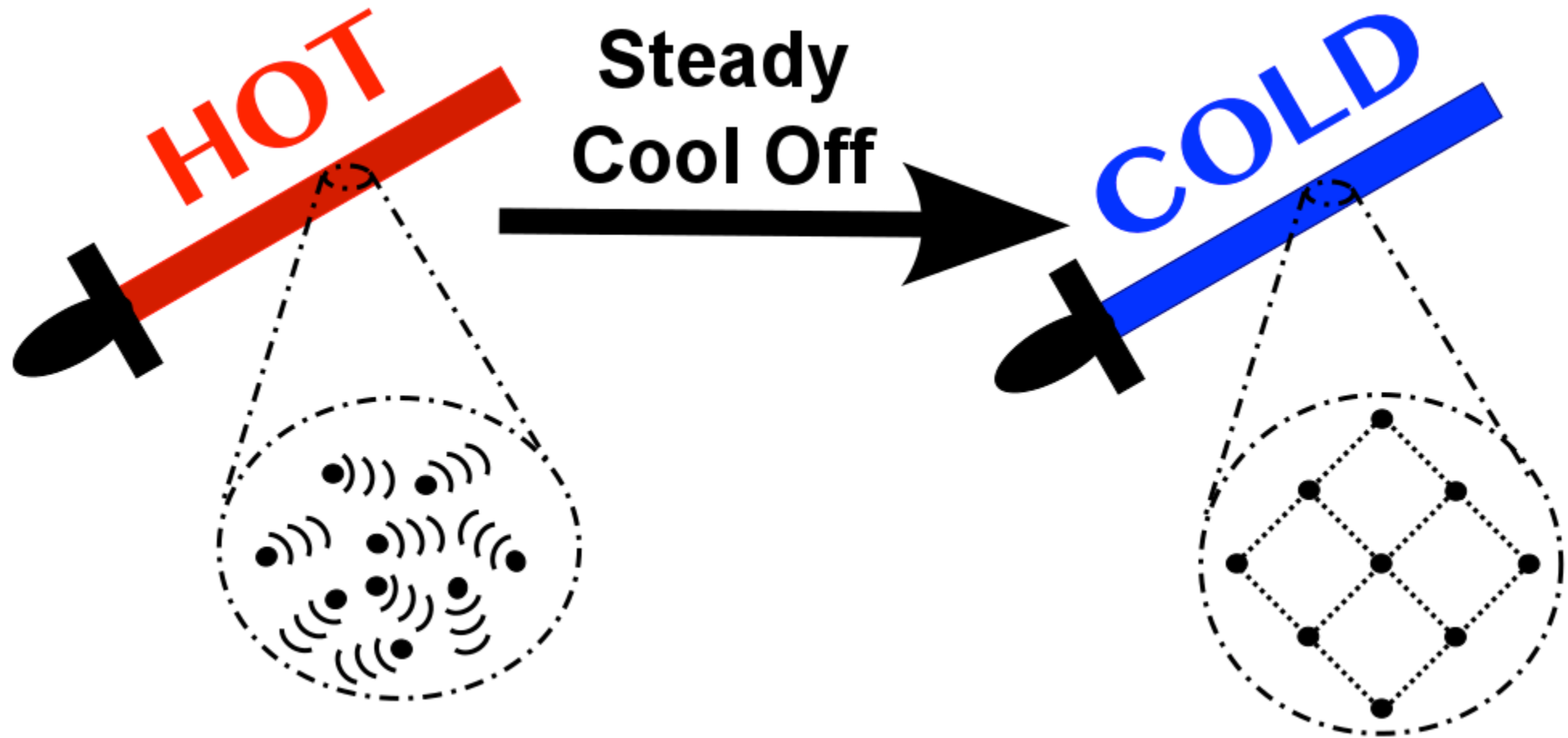
Simulated Annealing – Analogy

- If a liquid material **cools and anneals** too quickly, then the material will solidify into a **sub-optimal** configuration.
- If the liquid material **cools slowly**, the crystals within the material will solidify **optimally into a state of minimum energy** (i.e. ground state). – This ground state corresponds to the minimum of the cost function in an optimisation problem.



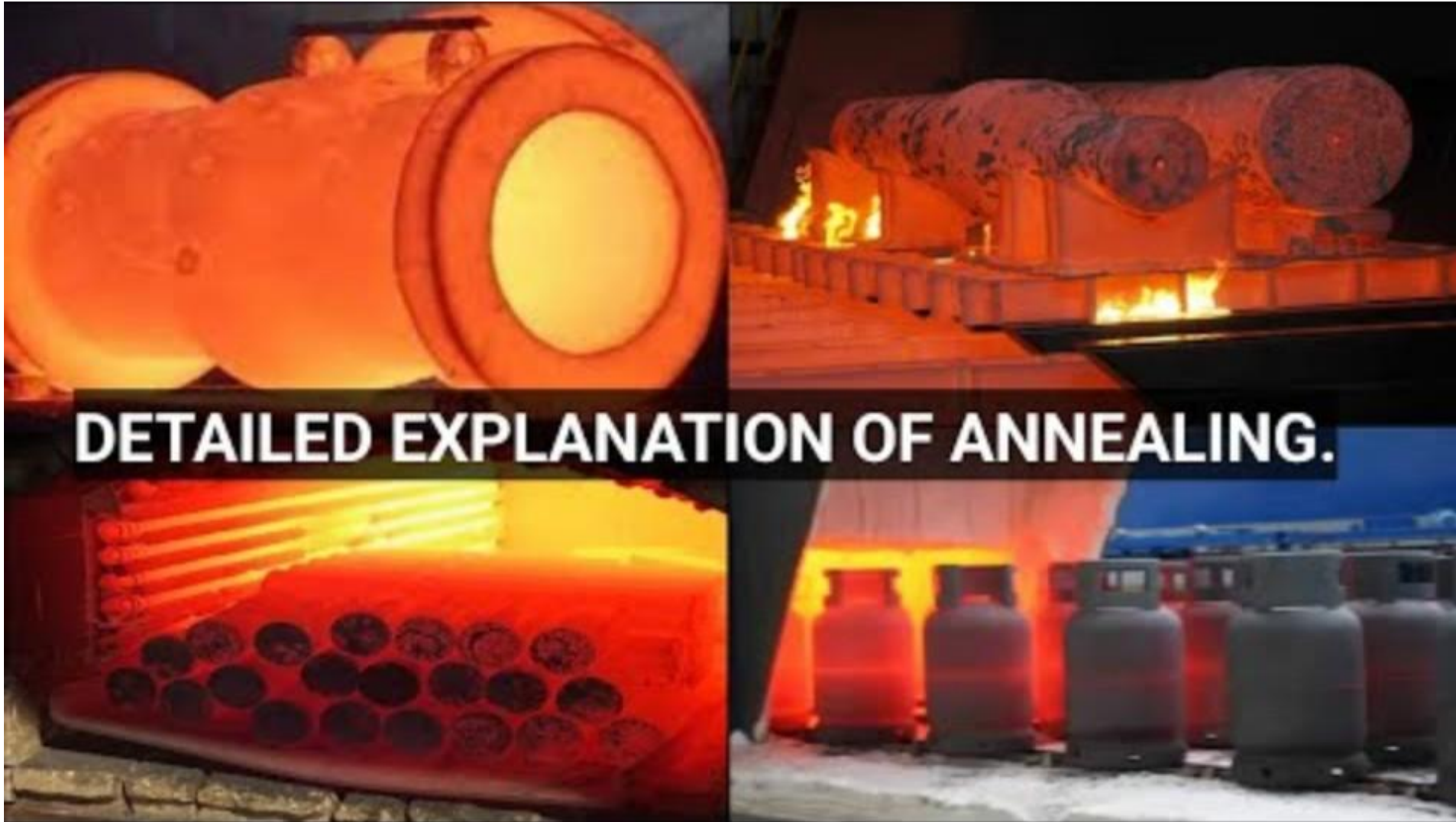


Simulated Annealing – Analogy





Annealing





Simulated Annealing – Analogy

Thermodynamic Simulation

Optimization Problem

States of system i

Solutions x_i

Energy of a state E_i

Cost of a solution $f(x_i)$

Change of a state

Neighbor of a solution

Temperature T

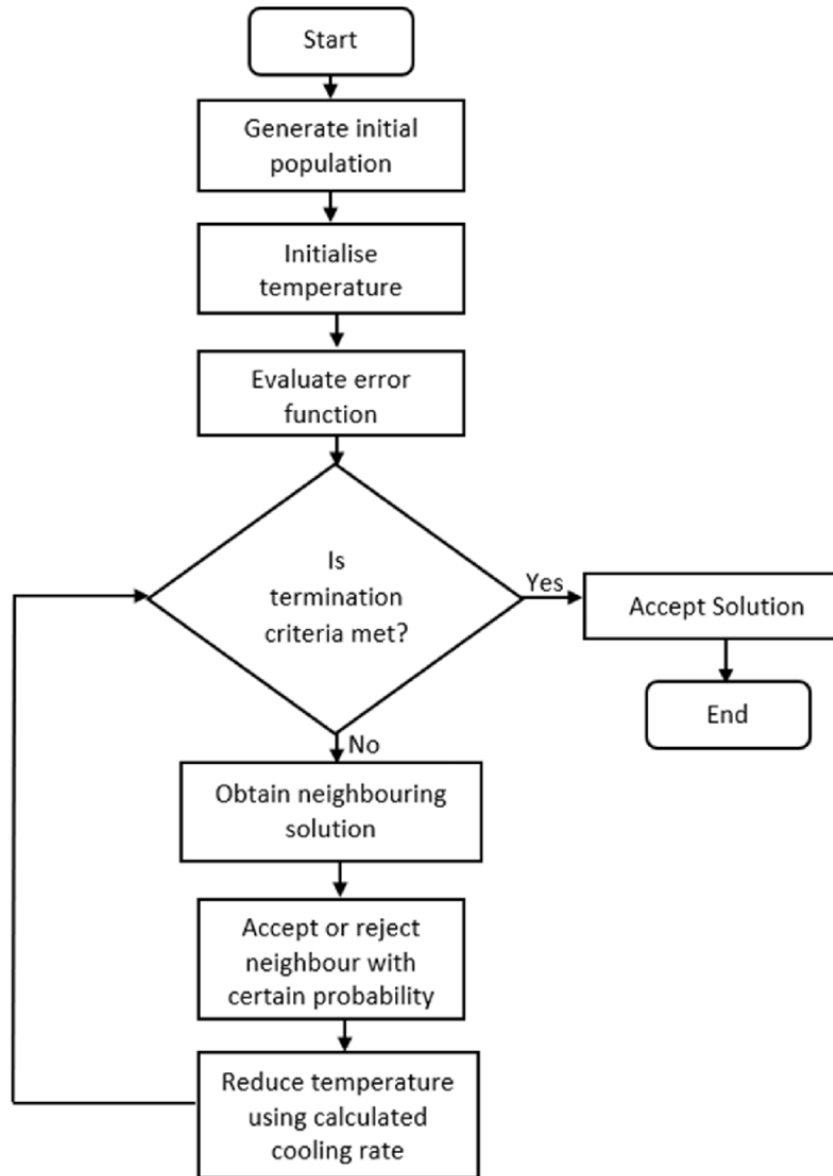
Control parameter T

Minimum energy E

Minimum cost $f(x)$



Simulated Annealing in General





Simulated Annealing Local Search Metaheuristic

INPUT: T_0, T_{final}

$s_0 \leftarrow generateInitialSolution$

$T \leftarrow T_0;$ // initialise temperature to T_0

$s_{best} \leftarrow s_0; s \leftarrow s_0;$ // set s and s_{best} to initial solution

REPEAT

$s' \leftarrow perturbation\ s;$ // choose a neighbouring solution of s

$\Delta = f(s') - f(s);$

$r \leftarrow random \in [0,1];$ // get a uniform random number in the range [0,1]

if($\Delta < 0 \ || \ r < P(\Delta, T)$) { // accept s' if solution is non-worsening or with Boltzmann probability

$s \leftarrow s';$

}

$s_{best} \leftarrow updateBest;$ // keep track of best solution

$T \leftarrow coolTemperature;$ // decrease the temperature according to cooling schedule

UNTIL (Termination conditions are satisfied);

Return $s_{best};$



Simulated Annealing Local Search Metaheuristic – Move Acceptance

if ($\Delta < 0$ or $r < \exp(-\Delta / T)$) then

$s \leftarrow s'$;

- The new solution s' is accepted in two cases:
 1. If the new solution is **better** (i.e., $\Delta < 0$), it is accepted.
 2. If the new solution is **worse**, it can still be accepted with a certain probability given by the expression $\exp(-\Delta/T)$. This *probabilistic* acceptance allows the algorithm to *explore* the solution space more freely at *higher temperatures*.
- Accepting new solutions based on the change in objective function and temperature to balance exploration and exploitation.



Simulated Annealing Local Search Metaheuristic – Cooling Schedule

- $T \leftarrow \text{coolTemperature}();$
- After each iteration, the temperature T is gradually decreased according to a *predefined cooling schedule*, which is crucial for the convergence of the algorithm.
- Gradually cooling the temperature to refine the search as the algorithm progresses.



Accepting Moves using SA

1. **Improving** moves are *accepted*.
2. **Worsening** moves are *accepted using the Metropolis criterion at a given temperature T* .

$$\Delta = F(S_{new}) - F(S_{old}) \quad \text{minimise}\{ F \}$$

An inferior solution S_{new} would yield $\Delta > 0$ (for minimization)

Accept it with a *Boltzman probability* of

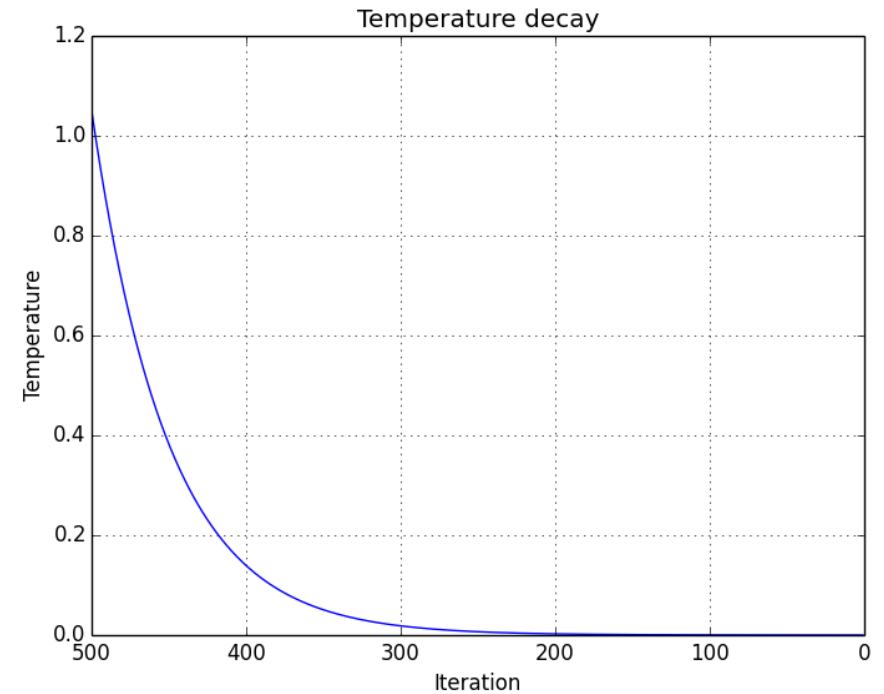
$$P(\Delta, T) = e^{\frac{-\Delta}{T}}$$

$U(0,1)$ generates a random number in $[0,1)$. Accept if $U(0,1) < P(\Delta, T)$



Boltzman Distribution

- **Boltzmann distribution** is a probability distribution or probability measure that gives the probability that a system will be in a certain state as a function of that *state's energy* and the *temperature* of the system.
- Boltzmann probability describes how likely a system is to be in a certain configuration based on its energy, and it follows the idea that at higher temperatures, the system is more likely to *explore higher-energy (worse)* configurations, while at lower temperatures, the system becomes more likely to *settle into lower-energy (better)* configurations.



Boltzmann's distribution is an exponential distribution.

$$p_i \propto \exp\left(-\frac{\epsilon_i}{kT}\right)$$

$$\frac{p_i}{p_j} = e^{-(E_i - E_j)/T}$$



Boltzmann Probability in Simulated Annealing

- The probability of accepting a move from state s_t to state s' (where $f(s)$ is the objective function, often representing the energy or cost of the state) is given by the Boltzmann distribution:

$$P(\text{accept}) = e^{-\frac{f(s') - f(s_t)}{T}}$$

Where:

- $f(s')$ is the cost (energy) of the new state.
- $f(s_t)$ is the cost (energy) of the current state.
- T is the current temperature.



Boltzmann Probability in Simulated Annealing

1. If the new solution s' is *better* than the current solution (i.e., $f(s') < f(s_t)$) (minimization), the probability of accepting it is high (since the exponent is negative).
2. If the new solution s' is *worse* than the current solution (i.e., $f(s') > f(s_t)$) (minimization), the probability of accepting it is still positive but decreases as the difference between the costs increases. At high temperatures, worse solutions are accepted more easily, whereas at lower temperatures, the algorithm becomes more selective.



Cooling

- In Simulated Annealing, the temperature decreases over time according to a cooling schedule. Temperature T is slowly decreased.
 - T is initially high - many inferior moves are accepted.
 - T is decreasing - inferior moves are nearly always rejected.
- **As the temperature T decreases, the probability of accepting worsening moves decreases.**

$\Delta > 0$ inferior solution

$$-\Delta < 0 \quad T \searrow \quad -\frac{\Delta}{T} \searrow \quad e^{\frac{-\Delta}{T}} \searrow$$



Boltzmann Probability: Example

Let's consider a simple example of **Simulated Annealing** to minimize the function $f(x) = (x - 3)^2$, where the goal is to find the minimum (which is at $x = 3$).

1. **Start with an initial solution:** Let's say $x = 0$ (i.e., $f(0) = (0 - 3)^2 = 9$).
2. **Propose a new solution:** Suppose the next candidate is $x' = 2$ (i.e., $f(2) = (2 - 3)^2 = 1$).
3. **Compute the change in energy:** The change in cost is:

$$\Delta E = f(2) - f(0) = 1 - 9 = -8$$

Since the new solution is better (lower energy), the move is accepted immediately.

4. **Now, let's propose a worse solution:** Suppose $x' = 4$ (i.e., $f(4) = (4 - 3)^2 = 1$).



Boltzmann Probability: Example

4. Now, let's propose a worse solution: Suppose $x' = 4$ (i.e., $f(4) = (4 - 3)^2 = 1$).
5. Compute the change in energy: The change in cost is:

$$\Delta E = f(4) - f(0) = 1 - 9 = -8$$

The change is negative, so we must calculate the Boltzmann probability of accepting this worse solution. Assume the current temperature is $T = 5$:

$$P(\text{accept}) = e^{-\frac{\Delta E}{T}} = e^{-\frac{-8}{5}} = e^{1.6} \approx 4.95$$

So, there is about a 70% chance of accepting the worse solution $x' = 4$.

As the temperature decreases, the probability of accepting worse solutions decreases, guiding the algorithm toward the optimal solution over time.



In Summary: Temperature and Boltzmann Probability

The **temperature** T plays a critical role in the Boltzmann probability:

- **High temperature:** At high temperatures, the system is more likely to accept worse solutions, as the exponent $e^{-\frac{\Delta E}{T}}$ approaches 1 for both good and bad moves. This enables the algorithm to explore the search space more freely.
 - **Low temperature:** At low temperatures, the system becomes more selective. The probability of accepting a worse solution (i.e., one with a higher cost or energy) diminishes, and the algorithm increasingly focuses on finding the best solution.
- This mimics the physical annealing process, where the system starts at a high temperature (allowing exploration of many states) and gradually cools down, settling into the optimal or near-optimal state.





SA – Cooling Schedule

- Starting Temperature
- Final Temperature
- Temperature Decrement (or Increment)
- Iterations at each temperature



SA – Cooling Schedule

Starting Temperature (T_0)

- *hot enough*: to allow almost all neighbours.
- *not so hot*: to allow random search for some time (not all the time)
- Estimate a suitable starting temperature:
 - Reduce quickly to 60% of worse moves are accepted.
 - Use this as the starting temperature.

Final Temperature

- Usually 0 (totally cooled), however in practice, not necessary.
- **T is low**: accepting a worse move is almost the same as $T=0$.
- The **stopping criteria**: either be a suitably low T , or “frozen” at the current T (i.e. no worse moves are accepted).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



SA – Cooling Schedule

■ Temperature Decrement (other than Boltzmann)

- **Linear:** $T = T - x$ (T is decreased by a fixed amount x in each iteration.)
- **Geometric:** $T = T * \alpha$ (slower cooling process. The choice of α is crucial)
 - Experience: α is typically in the interval $[0.9, 0.99]$.
- **Lundy Mees:**
 - One iteration at each T , but decrease T very slowly.
 - Experience:
 β is typically a very small value, that is close to 0 (e.g., 0.0001).

$$T_k = \frac{T_{k-1}}{1 + \beta T_{k-1}}$$

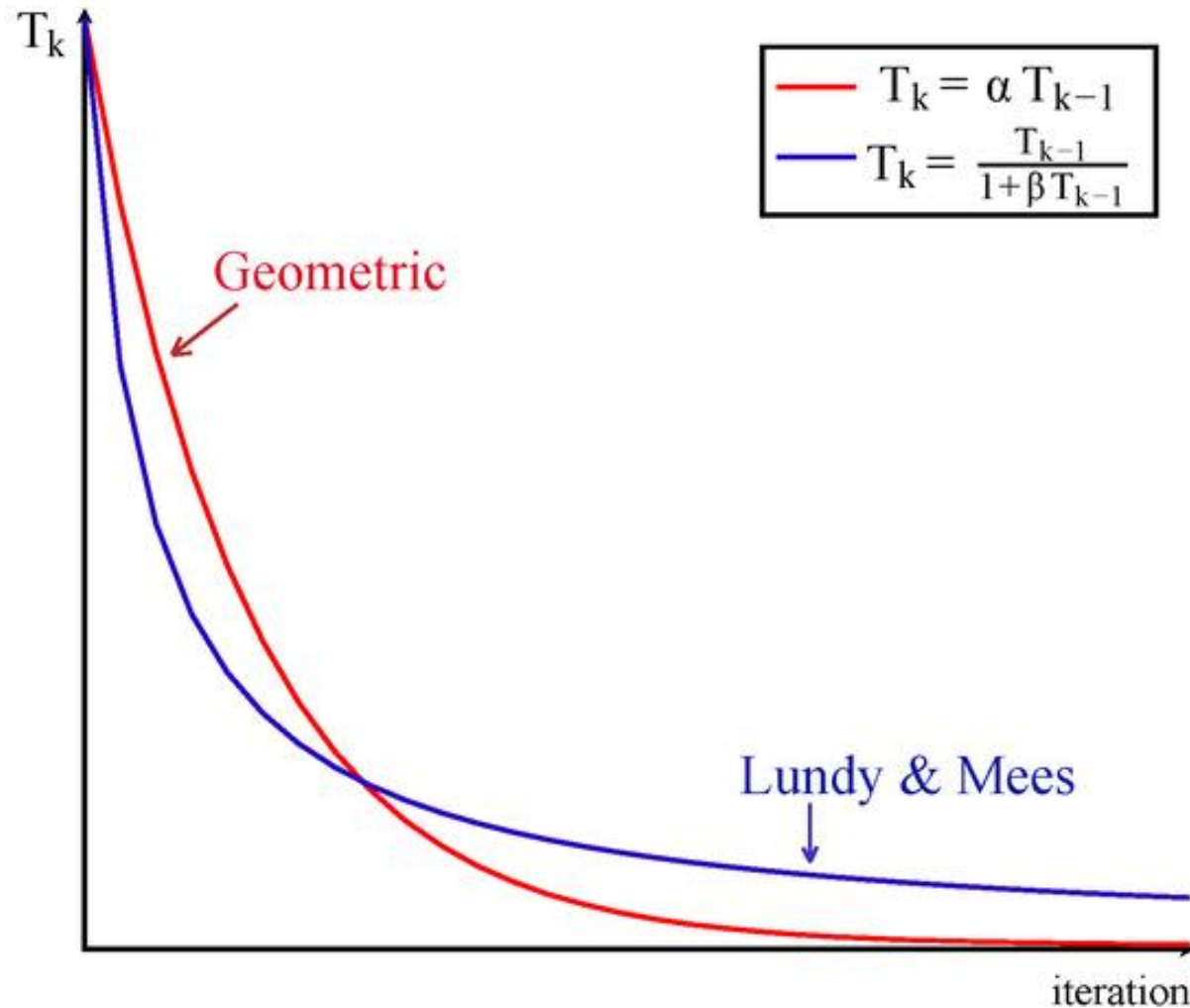


SA – Cooling Schedule

- **Linear Cooling:** Simple but may *cool too quickly*, risking premature convergence.
- **Geometric Cooling:** More controlled cooling that allows for *better exploration* as it decreases temperature exponentially.
- **Lundy-Mees Cooling:** *Adaptive* and slow cooling that can lead to better exploration of the solution space but requires careful tuning of parameters.



Geometric vs Lundy-Mees



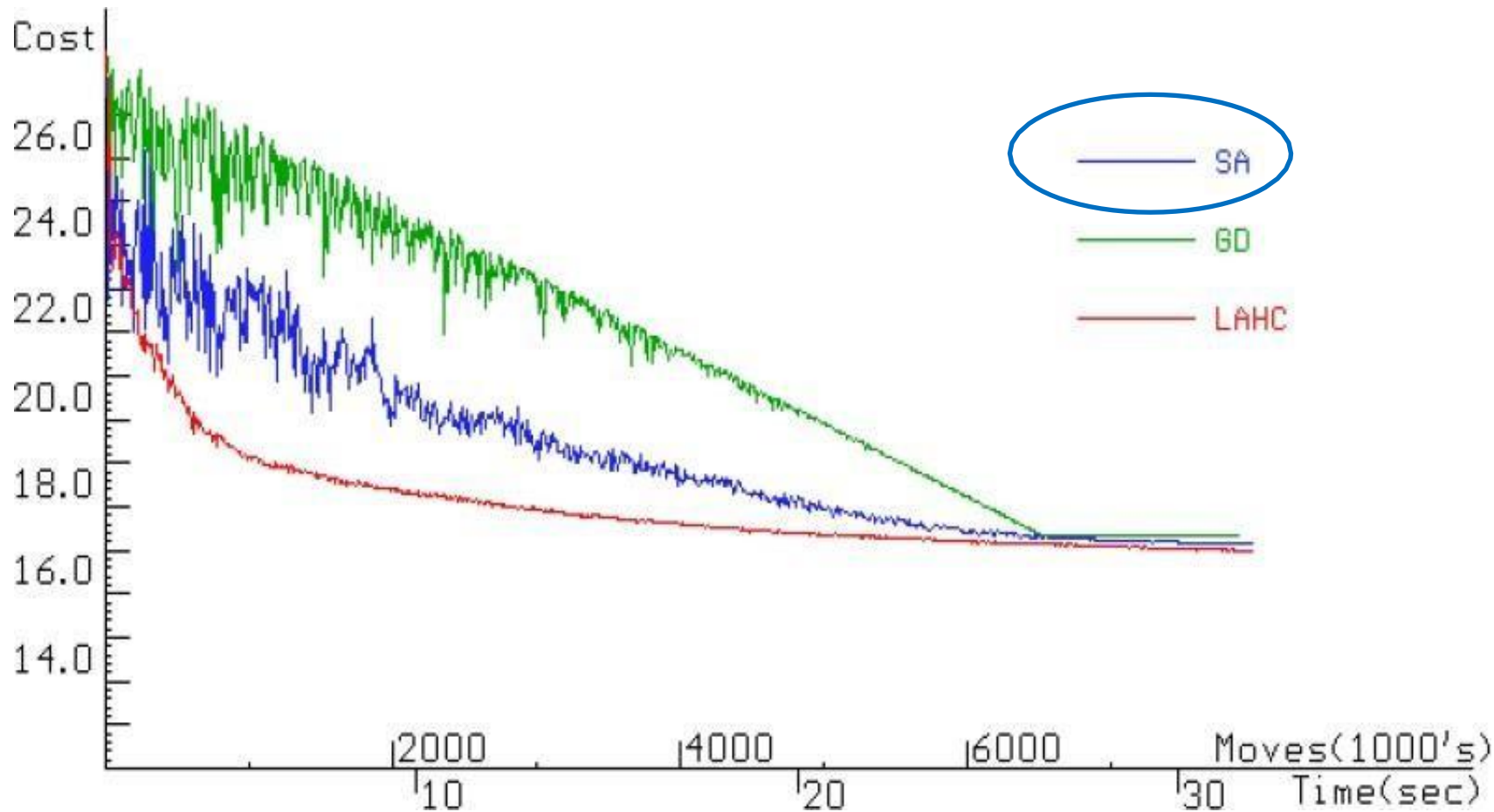


SA – Cooling Schedule

- Iterations at each temperature
 - One iteration at each T
 - A constant number of iterations at each T
 - Compromise
 - Either: a large number of iterations at a few T_s , or
 - A small number of iterations at many T_s , or
 - A balance between the two.
 - Dynamically change the no. of iterations.
 - At higher T_s : less no. of iterations.
 - At lower T_s : large no. of iterations, local optimum fully exploited.
- Reheating, if get stuck for a while increase the current.



Behaviour of Simulated Annealing – An Example



- **Green Curve (GD):** Gradient Descent
- **Blue Curve (SA):** Simulated Annealing
- **Red Curve (LAHC):** Local Adaptive Hill Climbing



Simulated Annealing with Geometric Cooling

INPUT: $T_0 (> 0)$, α (cooling rate < 1.0)

Generate an initial solution S_0 using some heuristics

Set $S_k = S_{best} = S_0$, $k=0$;

REPEAT

Select $S_{new} \in N(S_k)$ // Make a move from S_k to S_{new} based on N

If $F(S_{new}) < F(S_k)$ then $S_{k+1} = S_{new}$ // an improving move is made

else // A worsening solution is obtained

generate a random uniform number in $(0,1]$, $U(0,1)$

$$\frac{F(S_{new}) - F(S_k)}{T_k}$$

If $U(0,1) < e$ then $S = S$ // Accept worsening move

else $S_{k+1} = S_k$ // Reject worsening move

// Keep track of the best solution found so far

If $F(S_{new}) < F(S_{best})$ then $S_{best} = S_{new}$

$T_{k+1} = \alpha T_k$; // geometric cooling: multiply previous temp with α (value < 1.0)

$k = k+1$; // one iteration at each T

UNTIL (stopping condition = true)



Explanation: Simulated Annealing with Geometric Cooling

If $F(S_{new}) < F(S_k)$ then $S_{k+1} = S_{new}$

Else

- Evaluate the objective function F:
 - If $F(S_{new}) < F(S_k)$, the new solution is better, and it is accepted.
 - If $F(S_{new}) > F(S_k)$, the new solution is worse, and further evaluation is needed.



Explanation: Move Acceptance Condition

`r = generate a random uniform number in (0, 1);`

`if (0 < r < exp((F(Snew) - F(Sk)) / Tk)) then`

`Sk+1 ← Snew;   // Accept worsening move`

`else`

`// Reject worsening move`

- If the new solution is worse, it may still be accepted based on a probability that depends on the change in the objective function, $F(S_{\text{new}}) - F(S_k)$, divided by the current temperature T_k : The condition checks if the random number r falls within the acceptance probability, allowing for exploration of less favorable solutions at higher temperatures.



Simulated Annealing with Reheating

- **Reheating** refers to the strategy of *increasing the temperature after a cooling phase*, allowing the algorithm to explore the solution space more broadly again. This can help overcome stagnation and escape local optima that might be encountered during the cooling phase.
 - **Escaping Local Optima:** If the algorithm gets stuck in a local optimum during the cooling phase, reheating can help it jump out by allowing worse solutions to be accepted again.
 - **Enhanced Exploration:** Reheating provides additional opportunities for exploration, potentially leading to better overall solutions.
 - **Dynamic Adjustment:** It allows the algorithm to adapt to the landscape of the solution space dynamically.



How Reheating Works

- 1. Identify Stagnation:** Monitor the progress of the algorithm. If improvements stall for a certain number of iterations, it may indicate that the search is stagnating.
- 2. Increase Temperature:** Temporarily increase the temperature to a predefined reheating level, which may be slightly lower than the initial temperature.
- 3. Resume Exploration:** Continue the optimization process with the increased temperature, allowing for more exploration of the solution space.
- 4. Cool Down Again:** After a specified number of iterations at the reheated temperature, the algorithm can resume its cooling schedule.



Exercise – MAX-SAT

- Consider the MAX-SAT problem instance (CNF form: 6 literals, 6 clauses):
 $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$

Objective function: number of unsatisfied clauses (minimization).

- Apply the simulated annealing to the instance starting out with the 100100 as an initial solution for 3 SA steps/iterations.

Neighbourhood operator: perform random 1-bit flip.

- Choose $\alpha = 0.9$ and $T_0 = 0.9$
- Use the following numbers in the given order as **random numbers where appropriate**
 - for choosing a random literal i in $[1..6]$: $\langle 2, 3, 1, 6, \dots \rangle$ to apply 1-bit flip of (i)th literal's truth assignment in the candidate solution for $U(0,1)$: $\langle 0.47, 0.089, \dots \rangle$



Running of SA

$$\begin{array}{ccccc}
 0 & & 1 & & 2 \\
 (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \\
 \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \\
 3 & & 4 & & 5
 \end{array}$$

$$S_{best} = S_0 = 100100, T_0 = 0.9$$

$$F(S_0) = F(100100) = 3 \text{ (Clause SAT: } 0\textcolor{red}{1}234\textcolor{red}{5}) = F(S_{best})$$

k=0

$$S_{new} \leftarrow N(S_0): 110100 \quad \triangleright \text{Rand}[1..6]: \langle \textcolor{red}{2}, 3, 1, 6, \dots \rangle$$

$$S_{new} = 110100, \text{ Clause SAT: } 0\textcolor{red}{1}234\textcolor{red}{5}: F(S_{new}) = 4 > F(S_0) = 3$$

$$U(0,1) = 0.47 > e^{-\frac{4-3}{0.9}} = 0.33 \quad \triangleright U(0,1): \langle \textcolor{red}{0.47}, 0.089, \dots \rangle$$

$$S_1 = 100100$$

$\triangleright$ reject $S_{new}: S_1 = S_0$

$$F(S_{new}) = 4 > F(S_{best}) = 3$$

$\triangleright S_{best}$ does not change

$$T_1 = \alpha T_0 = 0.9 \cdot 0.9 = 0.81$$

$\triangleright$ update temperature

k=1

$$S_{new} \leftarrow N(S_1): 101100 \quad \triangleright \text{Rand}[1..6]: \langle 2, \textcolor{red}{3}, 1, 6, \dots \rangle$$

$$S_{new} = 101100, \text{ Clause SAT: } 0\textcolor{red}{1}2345: F(S_{new}) = 1 < F(S_1) = 3$$

$$S_2 = 101100$$

$\triangleright$ accept $S_{new}: S_2 = S_{new}$

$$F(S_{new}) = 1 < F(S_{best})$$

$$S_{best} = 101100$$

$\triangleright$ update best solution

$$\textcolor{red}{F(S_{best}) = 1}$$

$$T_2 = \alpha T_1 = 0.9 \cdot 0.81 = 0.729$$

$\triangleright$ update temperature



Running of SA

$$\begin{array}{ccccc}
 0 & & 1 & & 2 \\
 (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \\
 \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \\
 3 & & 4 & & 5
 \end{array}$$

k=2

$S_{new} \leftarrow N(S_2): 001100 \rightarrow \text{Rand}[1..6]: \langle 2, 3, \mathbf{1}, 6, \dots \rangle$

$S_{new} = 001100, \mathbf{01}2345, F(S_{new}) = 2 > F(S_2) = 1$

$U(0,1) = 0.089 < e^{-\frac{2-1}{0.729}} = 0.253 \rightarrow U(0,1): \langle \mathbf{0.089}, \dots \rangle$

$S_3 = 001100$

$\rightarrow \text{accept } S_{new}: S_3 = S_{new}$

$F(S_{new}) = 2 > F(S_{best}) = 1$

$\rightarrow S_{best}$ does not change (101100)

$T_3 = \alpha T_2 = 0.9 \cdot 0.729 = 0.6561$

...



Exercise Example

Step 1: Evaluate the Initial Solution

The initial solution **1001000** corresponds to:

- $a = 1$
- $b = 0$
- $c = 1$
- $d = 0$
- $e = 0$
- $f = 0$

Clauses Evaluation:

1. $(a \vee b) \rightarrow \text{True}$ (since a is true)
2. $(\neg d \vee f) \rightarrow \text{True}$ (since $\neg d$ is true)
3. $(\neg a \vee c) \rightarrow \text{True}$ (since c is true)
4. $(b \vee \neg f) \rightarrow \text{True}$ (since $\neg f$ is true)
5. $(\neg b \vee c \vee e) \rightarrow \text{True}$ (since c is true)

Total Satisfied Clauses: 5 out of 5



Exercise Example

Step 2: Generate New Solutions

We'll perform 3 iterations using a random 1-bit flip to generate new candidate solutions.

Iteration 1

1. **Random Number:** 2 (from the list)
2. **Bit Flip:** Flip the 2nd bit (b , which is currently 0).
3. **New Solution:** 1101000

Evaluate New Solution:

- $a = 1, b = 1, c = 1, d = 0, e = 0, f = 0$

Clauses Evaluation:

1. $(a \vee b) \rightarrow \text{True}$
2. $(\neg d \vee f) \rightarrow \text{True}$
3. $(\neg a \vee c) \rightarrow \text{True}$
4. $(b \vee \neg f) \rightarrow \text{True}$
5. $(\neg b \vee c \vee e) \rightarrow \text{True}$

Total Satisfied Clauses: 5 out of 5

Acceptance Probability:

- Since the new solution is equal to the current best, we accept it.



Exercise Example

Step 3: Update Temperature

- **New Temperature:**

$$T_1 = \alpha \times T_0 = 0.9 \times 0.9 = 0.81$$

Step 4: Iteration 2

1. **Random Number:** 3 (from the list)
2. **Bit Flip:** Flip the 3rd bit (c , which is currently 1).
3. **New Solution:** 1100000

Evaluate New Solution:

- $a = 1, b = 1, c = 0, d = 0, e = 0, f = 0$

Clauses Evaluation:

1. $(a \vee b) \rightarrow \text{True}$
2. $(\neg d \vee f) \rightarrow \text{True}$
3. $(\neg a \vee c) \rightarrow \text{False}$
4. $(b \vee \neg f) \rightarrow \text{True}$
5. $(\neg b \vee c \vee e) \rightarrow \text{False}$

Total Satisfied Clauses: 3 out of 5



Exercise Example

Acceptance Probability:

- Current: 5, New: 3
- $\Delta E = 3 - 5 = -2$
- Acceptance Probability:

$$P = \exp\left(\frac{-\Delta E}{T_1}\right) = \exp\left(\frac{2}{0.81}\right) \approx 6.7$$

- Since $P > 1$, we accept the new solution.

Step 5: Update Temperature

- **New Temperature:**

$$T_2 = \alpha \times T_1 = 0.9 \times 0.81 = 0.729$$



Exercise Example

Step 6: Iteration 3

1. **Random Number:** 1 (from the list)
2. **Bit Flip:** Flip the 1st bit (a, which is currently 1).
3. **New Solution:** 0100000

Evaluate New Solution:

- $a = 0, b = 1, c = 0, d = 0, e = 0, f = 0$

Clauses Evaluation:

1. $(a \vee b) \rightarrow \text{True}$
2. $(\neg d \vee f) \rightarrow \text{True}$
3. $(\neg a \vee c) \rightarrow \text{True}$
4. $(b \vee \neg f) \rightarrow \text{True}$
5. $(\neg b \vee c \vee e) \rightarrow \text{False}$

Total Satisfied Clauses: 4 out of 5

Acceptance Probability:

- Current: 3, New: 4
- Since this new solution is better, we accept it.



Exercise Example

- After 3 iterations, we have explored different solutions with varying degrees of satisfaction. The best solution found is 1101000 with 5 satisfied clauses. Depending on the cooling schedule and random flips, further iterations could continue to explore other candidate solutions to find potentially better results.
- The process illustrates how Simulated Annealing can navigate through the solution space effectively while balancing exploration and exploitation.



University of
Nottingham

UK | CHINA | MALAYSIA

Example: Traveling Salesman Problem (TSP) with Simulated Annealing



Traveling Salesman Problem (TSP) with Simulated Annealing

Consider the following cities and distances:

City	A	B	C	D	E
A	0	10	15	20	25
B	10	0	35	25	30
C	15	35	0	30	5
D	20	25	30	0	15
E	25	30	5	15	0

1. Initialize

- **Initial solution:** Random route, e.g., $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$.
- **Initial temperature:** $T_0 = 100$
- **Cooling rate:** $\alpha = 0.95$
- **Stopping condition:** Stop after 100 iterations.



Traveling Salesman Problem (TSP) with Simulated Annealing

2. Iteration Process

1. Start with the initial solution.
2. Generate a neighboring solution (e.g., swap two cities in the route).
3. Calculate the cost (total distance) for both the current and new solutions.
4. If the new solution is better, accept it. If it's worse, accept it with a probability based on the temperature and cost difference.
5. Lower the temperature and repeat.

3. Example Walkthrough

1. **Start with route:** $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$ with a distance of $10 + 35 + 30 + 15 + 25 = 115$.
2. **Generate neighbor:** Swap **B** and **C**: $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow A$. The new distance is $15 + 35 + 25 + 15 + 25 = 120$.
3. Since the new solution is worse, calculate the acceptance probability $P = e^{-(120-115)/100} = e^{-0.05} \approx 0.951$. The solution is accepted with a probability of 95.1%.
4. **Decrease temperature:** $T_1 = 0.95 \times 100 = 95$.
5. Repeat the process until the stopping condition is met.



Summary

- **Move acceptance** methods as a part of local search metaheuristics can be used for escaping from the local optima, enabling acceptance of non-improving solutions.
 - **Great Deluge** and **Simulated Annealing** are well-known and well-studied such local search metaheuristics, and recently **Late Acceptance**. These are easy to implement methods and there are more elaborate variants.
 - **Move acceptance methods** vary depending on the mechanism and components they have for accepting a non-improving solution.



Summary

- Move acceptance methods can be **hybridised**:
 - **Single stage strategy**:
 - Use a decision mechanism to choose which move acceptance to employ at each step.
 - Use group decision making. For example, combine simulated annealing, great deluge and late acceptance and apply majority vote.
 - **Multi-stage**
 - Use a different move acceptance at different stages of the search process.
 - For example, use improving and equal until the search process gets stuck, then switch to simulated annealing.



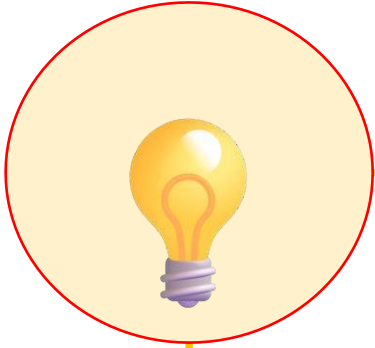
Summary

- It is not trivial which (meta)heuristic optimisation/search algorithm will perform better than the other one on a given problem domain.
 - **Experiments** using a single or only small instances might not be realistic or true performance indicator of an algorithm.
- There is a variety of statistical tools for the **performance analyses of algorithms**.



Summary

- Many (meta)heuristic optimisation/search algorithms come with **parameters** which often require an initial setting (tuning).
 - **Parameter tuning** is possible; however, it is time consuming.
 - There is a range of different techniques varying from *manual/semi-automated experimental design* methods to automated tuning, such as, Taguchi method, I- race.
 - **Parameter control** as an alternative to parameter tuning changes parameter values during the run of the algorithm.
 - There is no guidance indicating which method is the best, however many studies show that parameter tuning/control often does improve the performance of an algorithm as compared to the **variant where it is not used.**



1. Move Acceptance
2. Simulated Annealing
3. **Parameter Setting Issues**





University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

**Evolutionary
Algorithm I**